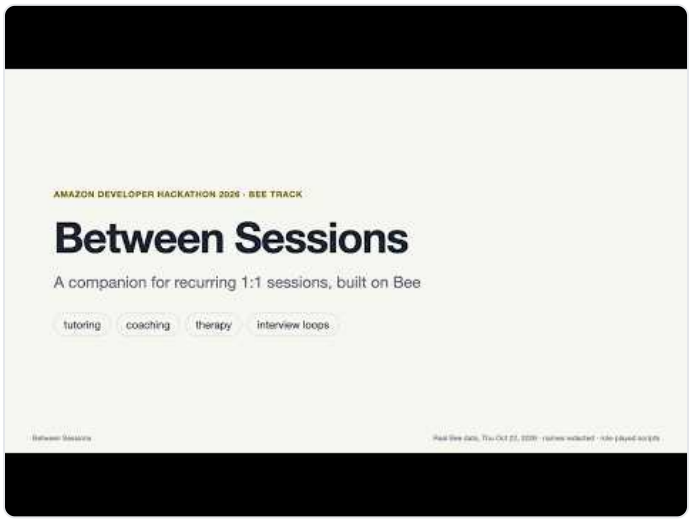
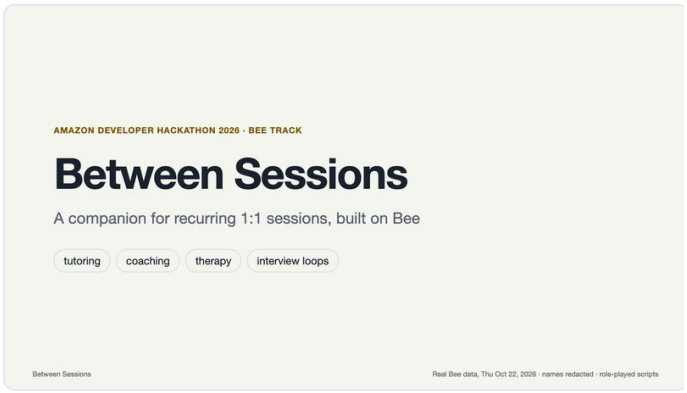
Captured from the public project page https://devpost.com/software/between-sessions on Fri Oct 23, 2026. Text is as published; the page's navigation, buttons and comments are left out.

Between Sessions

Between Sessions files what your Bee heard into the recurring sessions that matter (tutoring, coaching, therapy), on Bee's free tier, in your AWS account, and you talk to it through Bee's own button.





Creepy, but amazing!

Demo video: https://www.youtube.com/watch?v=Ecn05U63wMc

Inspiration

It's Thursday, 5:55. You're about to walk into your tutoring session. What did you want to bring up? What did you take away last time? What has come up since?

Recurring one-on-ones (tutoring, coaching, therapy, an interview loop) live or die on the week around them. The hour itself is easy. The thing you meant to raise on Tuesday is gone by Thursday. And a Bee wearable already heard all of it.

An always-listening wearable felt a little creepy, so I wanted the opposite of a digital twin: a narrow tool for a few recurring sessions, with my data in my own AWS account. On Bee's free tier there's no calendar integration, and Bee keeps about two weeks of data. Its useful summaries arrived overnight, in an app I had to remember to open. Between Sessions copies just the sessions that matter into a store I control, and brings them back right before the next session. The schedule and the Bee button stand in for a calendar.

What it does

Between Sessions turns Bee's ambient record into a companion for one recurring session at a time.

- **Before a session**, ask for a prep brief and get:
  - the last session and Bee's title for it;

    - what came up since (days, conversations, topics from last time that came back);
    - your open agenda and your debrief;
    - when the next session is.
- **During the week**, it finds your sessions in Bee's conversations on its own. It uses evidence in a fixed order: what you confirmed, Bee button markers, your schedule (a weekly slot or dated slots), then a keyword in Bee's own summary. Each conversation lands in at most one session.
- **It never guesses silently.** A match it isn't sure of, say the right slot but the wrong topic, waits in a review queue with Confirm, Move and "Not a session". Summaries stay hidden on the review pages until you choose to show them.
- **Bee's button is an input.** Hold it and talk to the product by name, Marty:
    - "Marty, agenda for tutoring: bring up the essay deadline."
    - "Marty, debrief: that went better than last week."
    - "Marty, start finance session" … "Marty, end finance session."

Voice notes are READY in 3 to 6 seconds, so this is a real-time command channel. A start note can even create a new series for an unscheduled meeting. Notes Marty can't read are reported in the next brief, never dropped. The name matters: a note addressed to Marty is a command, and anything else stays a plain note.

- **Ask from any MCP client.** Eleven MCP tools (prep brief, since last session, agenda, takeaways, review, retag, help) and six prompts answer from precomputed data, with no model call on the request path.

How we built it

- **A collector Lambda** runs every 15 minutes on EventBridge Scheduler. It ships the Bee CLI's Linux arm64 binary (`@beeai/cli`), restores the CLI login from Secrets Manager, and runs `bee conversations list`, `bee insights list`, `bee journals list` and `bee todos list`. Then it places sessions, files voice notes, updates the review queue and precomputes one digest per series in DynamoDB.
- **An MCP Lambda** behind API Gateway (Streamable HTTP, bearer token) answers from DynamoDB only.
- **A local watcher** follows `bee stream` and shows each voice note's verdict within seconds of speaking. Two local pages show every stored session, marker, voice note and review item, plus a Schedule editor.
- **One CDK stack** in your own AWS account: the table is retained with deletion protection, there's an error alarm on the collector, and API Gateway is throttled.
- **226 tests** with a fake Bee CLI. Built in four days with Claude Code as the development agent.

Challenges we ran into

- Bee closes conversations 13 to 27 minutes after talk ends and merges talk across short gaps, so slot matching counts minutes inside the slot, not whole conversations.
- An open conversation has no end time, so nothing is filed until Bee closes it. A double-press on the Bee button closes it early.
- Bee's voice-note transcription drops punctuation, and its cleaned-up text drops a leading name, so the command parser works from raw speech and needs the name Marty.
- The Bee CLI stalled about 20 seconds per call on the macOS Keychain. An undocumented file store fixed that, and also made the CLI work in Lambda.

- Bedrock's Nova quotas were 0 until Thursday evening, so an Alexa voice front end was cut for this submission.

Accomplishments that we're proud of

It works on a real Bee. On Thu Oct 22 we ran role-played scripts against tutoring slots, with no button presses for the scheduled ones:

- **Two 5-minute slots filed automatically** at 17:30 and 18:00. Bee's conversations ran 34 to 40 minutes, but only the minutes inside each slot count.
- **An off-topic conversation in the 19:00 slot** (planning a hike) went to review instead of being filed.
- **A canceled 21:00 weekly sync stayed empty**, even with a conversation that started at 21:06. It covered 24 of the slot's 30 minutes, under the series' minimum.
- **An agenda item** said into the Bee button was filed within seconds. An unscheduled Finance meeting was started and ended by voice and filed as its own session.

Also:

- **Bee's own button became a real-time command channel** for another product, with no app to open.
- **A 26-entry friction log** written as we went, with a workaround and a suggestion for each entry.

What we learned

- **Start with the realtime stream.** I spent my first 6 hours learning the CLI against three weeks of my own data. `bee stream` would have shown sooner how conversations, insights and voice notes are created and completed.
- **Design around Bee's two speeds.** Voice notes are READY in 3 to 6 seconds, fast enough to be a command channel. Conversations close 13 to 27 minutes after talk ends. The product uses the fast path for commands and is patient with the slow one.
- **A name makes voice commands reliable.** Addressing notes to Marty keeps commands apart from ordinary notes, and Bee's transcription drops the punctuation a "Command: …" grammar would need.
- **Ambient data earns trust by asking.** Filing only what the evidence supports, and sending the rest to review, mattered more than clever matching.
- **Narrow beats broad.** A laser focus on a few recurring sessions was easier to build, easier to trust, and more useful than a digital twin.
- **Request cloud model quotas on day one.** On a new AWS account the Bedrock quotas started at 0, and lifting them took about 36 hours through AWS Support.

What's next for Between Sessions

- **Bee captures, Alexa answers:** "Alexa, ask Between Sessions to prep me for tutoring." The MCP server already answers Claude Code, and the Nova quota for the Alexa bridge landed Thursday evening, so the bridge can resume.
- **Spoken review:** confirm or retag a guess by voice, with date, time and length only, never the summary.
- **A small web app with sign-in**, to replace the local review pages.

Creepy, but amazingly useful.

Built With

amazon-api-gateway | amazon-cloudwatch | amazon-dynamodb | amazon-eventbridge-scheduler | aws-cdk | aws-lambda | aws-secrets-manager | bee-cli | bee-realtime-stream | claude | model-context-protocol | node.js | typescript

Try it out

GitHub Repo:
https://github.com/wilmccann/between-sessions

Created by

FTS-William McCann

Submitted to

Build, Ship, Shape: Amazon Developer Hackathon

Appendix: submission form answers

Read from the submission's "Additional info" step on Devpost, as saved on Fri Oct 23, 2026 at about 2:45 PM ET. These answers go to the organizers and judges and aren't shown on the public project page. Personal and eligibility fields are left out.

Tracks and requirements

- **Primary track(s):** Bee
- **Code repository:** https://github.com/wilmccann/between-sessions
- **New or existing prior to August 31, 2026:** New
- **AWS Builder Mini Challenge:** Yes
- **Open Source Mini Challenge:** No
- **Uploaded file:** between-sessions-submission.zip
- **Friction log:** https://github.com/wilmccann/between-sessions/blob/main/docs/FRICTION_LOG.md
- **Project testing link:** https://github.com/wilmccann/between-sessions/blob/main/docs/TESTING.md
- **Testing instructions:** https://github.com/wilmccann/between-sessions/blob/main/docs/TESTING.md

AWS Builder Mini Challenge: Which AWS services did you incorporate and how?

Between Sessions runs entirely in the user's own AWS account, defined in AWS CDK (TypeScript). One stack holds the app (`infra/stack.ts`) and a second holds cost guardrails (`infra/guardrails.ts`). The pipeline: Bee wearable → Bee CLI in Lambda → DynamoDB → MCP server.

- **AWS Lambda (Node 22, arm64), two functions.**
  - The **collector** ships the Bee CLI's Linux binary inside the function and runs `bee conversations / insights / journals / todos list --json`. It finds recurring sessions in Bee's conversations, files Bee button voice commands, updates the review queue, and precomputes a digest per series.
  - The **MCP server** is a stateless Streamable HTTP endpoint that answers 11 tools from DynamoDB only, with no model call on the request path.
- **Amazon EventBridge Scheduler** runs the collector every 15 minutes, so a finished Bee conversation reaches the user's account within about 7.5 minutes on average.
- **Amazon DynamoDB**, one on-demand table, holds versioned sessions, agenda items, takeaways, debriefs, Bee button markers, voice-note outcomes, digests and the review queue. Point-in-time recovery and deletion protection are on, and the table is retained if the stack is deleted. Since Bee's free tier keeps about two weeks of data, this is the user's own lasting copy.
- **AWS Secrets Manager** holds the Bee CLI login. It's restored into the function's `/tmp` on each run and saved back when the CLI refreshes its token. It also holds the MCP bearer token. No secret is ever in code or logs.
- **Amazon API Gateway (HTTP API)** exposes `POST /mcp` to Claude Code or any MCP client, throttled to 2 requests per second (burst 5).

- **Amazon CloudWatch** keeps log groups with two-week retention and an error alarm on the collector.
- **Cost guardrails**:
  - AWS Budgets (daily and monthly);
  - Cost Anomaly Detection;
  - Amazon SNS email alerts;
  - a kill-switch Lambda that sets the app's functions to zero concurrency if the daily budget is hit.
- **Amazon Bedrock:** a Converse integration for session notes and prep briefs is built and tested, but ships switched off. On-demand Nova quotas were 0 on our new account until AWS Support lifted them, about 36 hours later. With the quota now in place, the next step is an Alexa voice front end on Nova 2 Lite.

The whole project was built with Claude Code as the development agent, working under repo rules that keep Bee data out of the repo and secrets out of the agent's reach.

Feedback Question 1: Which developer tools, APIs, and SDKs did you use and for what?

Bee

- **Bee CLI (`@beeai/cli`, an npm dependency).** All Bee data in code. The collector Lambda ships the CLI's Linux arm64 binary and runs `bee conversations list`, `bee insights list`, `bee journals list` and `bee todos list` (`--json`, paged with `--cursor`) every 15 minutes. The local review page reads the same lists live, plus `bee search` (keyword and semantic).
- **Bee realtime stream (`bee stream --json`).** A local watcher shows each Bee button voice note's verdict within seconds ("Marty, start session" → START).
- **Bee voice notes (journals) and the Bee button.** The input channel: hold the button and talk to the product by name. Double-press closes the current conversation.
- **Bee Android app.** Pairing, Developer Mode, and the button's double-press setting.

AWS

- **AWS CDK (TypeScript).** Two stacks: the app, and cost guardrails.
- **AWS Lambda.** The collector, the MCP server, and a cost kill switch.
- **Amazon EventBridge Scheduler.** The 15-minute collector schedule.
- **Amazon DynamoDB.** Sessions, agenda, takeaways, debriefs, markers and the review queue.
- **AWS Secrets Manager.** The Bee CLI login and the MCP token.
- **Amazon API Gateway (HTTP API).** `POST /mcp`.
- **Amazon CloudWatch.** Logs and an error alarm.
- **AWS Budgets, Cost Anomaly Detection and Amazon SNS.** Spend alerts.
- **Amazon Bedrock (Converse API).** Session notes and prep briefs: built, but switched off pending quota.
- **AWS SDK for JavaScript v3.** DynamoDB, Lambda, Secrets Manager, SNS and Bedrock Runtime clients.

Other

- **Model Context Protocol TypeScript SDK.** The MCP server (11 tools, 6 prompts, Streamable HTTP).
- **Claude Code.** The MCP client in the demo, and the development agent for the whole build.

- **Alexa+ MCP Toolkit.** Tried, to bring the MCP server to Alexa. Blocked by preview access.
- **TypeScript, Node.js 22, zod, esbuild and tsx** for the code; **gitleaks** for secret scanning before each push.

Feedback Question 2: For each tool, API, or SDK used in your project, what worked well?

Bee CLI (`@beeai/cli` 0.7.3 to 0.8.0)

- The data model fits this kind of app: conversations with start and end times, short summaries, insights, voice notes (journals) and todos, all as JSON with cursors.
- The Linux arm64 binary ran inside AWS Lambda unchanged, once it had a file-store login.
- `bee search` (keyword and semantic) worked from the same CLI, with no extra setup.

Bee realtime stream and voice notes

- Voice notes are fast: `PREPARING` to `READY` in 3 to 6 seconds, measured on 40 notes. That speed is what made "talk to the product through the Bee button" possible.
- `bee stream --json` is one long-lived process with clean JSON lines, so a watcher was easy to build on it.
- The single button with a configurable double-press is a simple, reliable input.

Bee docs and REST API: the conversation and journal shapes were documented well enough to build against.

AWS

- CDK made the whole backend reproducible in one `cdk deploy`.
- Shipping a third-party CLI binary inside Lambda, with its login in Secrets Manager, worked cleanly.
- EventBridge Scheduler plus a precomputed DynamoDB table kept MCP answers fast (well under a second) and cheap.
- The Bedrock Converse API was straightforward to code and test against a fake.

MCP TypeScript SDK and Claude Code

- Stateless Streamable HTTP with JSON responses fits Lambda well.
- MCP prompts show up as slash commands in Claude Code, which makes a natural menu.
- Claude Code as the development agent took the project from idea to a deployed, tested backend in four days.

Feedback Question 3: For each tool, API, or SDK used in your project, what needs work?

Details and entry numbers are in the friction log: https://github.com/wilmccann/between-sessions/blob/main/docs/FRICTION_LOG.md

Bee CLI

- **Speed on macOS:** about 20 seconds per authenticated call, from a Keychain stall (#11). `BEE_CONFIG_DIR` and `BEE_FORCE_FILE_STORE` fix it but appear only in the binary's strings. The ad-hoc signed macOS binary also asks for the Keychain password once per process (#24).

- **No time filters** on list commands or MCP tools (#13). Reading one weekly slot means paging through everything, newest first.
- **Limits and help text:** `insights list --limit 200` fails with only "Bad Request"; 100 is the unstated maximum (#25). `--cursor` is missing from the insights help (#14). Data commands need `list` while `today` and `me` don't (#9).
- **Server use is undocumented:** how to get a token for a backend (#5), which API host to use (#6), how to run headless (#17). The npm package pulls every platform's binary, about 430 MB (#17).
- `bee status` prints part of the token (#12).

Bee realtime stream and voice notes

- Events have no `event` field, so they're told apart by their keys (#18), and they can repeat (#19).
- Transcription drops punctuation (#21), and `cleanedUpText` drops a leading name, turning "Marty, start session" into a bare command (#22).

Bee docs and REST API

- No server-side token story (#5), and two API hosts with no guidance (#6).
- Forwarded endpoints such as insights and today's brief are listed by name only (#7, #16).
- What `search` indexes, and how its time bounds apply, isn't stated (#15). Typical voice-note latency isn't stated (#23).
- No list of built-in Actions (#3).

AWS (Bedrock)

- On a new account, every Nova tokens-per-day quota was 0 and not adjustable in Service Quotas, and Claude models returned "not available for this account" (#26). Only a Support case could lift the quota, which took about 36 hours and cost us the Alexa voice front end.
- It isn't obvious which quota a Converse call through an inference profile uses.

Alexa+ MCP Toolkit: `npm install -g @alexa-ai/cli` returns 404, and setup needs preview access through an Alexa Solutions Architect, with no stated way to request it (#10).

MCP SDK and Claude Code: nothing blocking for this project.

Feedback Question 4: For each tool, API, or SDK used in your project, how was your onboarding experience?

Bee (device, Android app, CLI): slow, then quick.

- Developer Mode and `bee login` pairing couldn't be done from the Android app until a device was paired (#1, #4), and the docs describe iOS only (#2). These were S1 blockers.
- Once paired, `bee login` and the first `bee me` took minutes.
- The Pioneer arrived two weeks into the hackathon. I spent the first 6 hours learning the CLI against 3 weeks of my own data. In hindsight I'd have started with the realtime stream, to see how conversations, insights and voice notes are created and completed.

Bee in the cloud: about an hour lost getting the CLI to run headless and in Lambda: 45 minutes on the Keychain stall and its undocumented file store (#11), and 20 minutes on the 430 MB package (#17).

AWS (CDK, Lambda, DynamoDB, Scheduler, Secrets Manager, API Gateway): quick. The CDK stack was written and deployed on the first night, and the Lambda go/no-go passed the next morning.

Amazon Bedrock: blocked at hello world by a 0 quota on a new account (#26). Lesson: request model quotas on day one.

MCP SDK and Claude Code: quick. A local MCP server answering from real Bee data was working within an hour of starting.

Alexa+ MCP Toolkit: didn't get to hello world; blocked by preview access (#10).

Feedback Question 5: Would you build with these devices and services again?

Yes. The Bee Pioneer is a capable ambient AI with a helpful CLI to build on, and its voice notes are fast enough to be a real command channel. The AWS serverless stack made it easy to keep my own copy of the data that matters, in my own account, before Bee's free-tier window rolls past it. It's still a little creepy, but so is AI in general: creepy, but amazingly useful.

[Optional] Feature Requests

1. **Time filters on Bee list commands and MCP tools** (Critical). Add `since` and `until` to `conversations list`, `insights list` and `journals list`, and to the matching MCP tools. *Why it matters:* Between Sessions reads one weekly or dated slot at a time. Without a time filter, every run pages through everything newest first and filters locally, which is slow and wasteful from a Lambda function every 15 minutes.
2. **Documented server-side access** (Critical). Document how to get a token for a backend, which API host to use from servers, and how to run the CLI headless (file store, config directory, token refresh). *Why it matters:* our collector runs the CLI in AWS Lambda. We got there by reading the binary's strings for `BEE_CONFIG_DIR` and `BEE_FORCE_FILE_STORE`. Every cloud app on Bee needs this.
3. **A push event when a conversation or voice note completes** (Important). A webhook, or a server-friendly stream, for `conversation completed` and `journal ready`. *Why it matters:* today a cloud backend has to poll. A push would file a session the moment Bee finishes it, instead of up to 15 minutes later.
4. **A free-tier way to tag a conversation to a recurring meeting** (Important). A documented tag or metadata field on conversations, settable from the button or the API, that doesn't need the calendar integration. *Why it matters:* on the free tier there's no calendar, so we infer meetings from a schedule the user types in, plus voice markers. A tag would make that exact.
5. **Keep a leading name in `cleanedUpText`, and document what cleanup changes** (Important). *Why it matters:* our voice commands are addressed by name ("Marty, start session"). The cleaned-up text drops "Marty," and turns a command into what looks like an ordinary note, so we have to parse the raw text instead.
6. **An `event` field on every `bee stream` payload** (Important). *Why it matters:* our watcher tells events apart by which keys they have, and the first message after connecting is undocumented. A type field would make realtime apps simpler and safer.

7. **"Close the current conversation" for apps** (Nice-to-have). An API call or stream event matching what the button's double-press does. *Why it matters:* an app could end a marked session's conversation exactly at "Marty, end session", instead of asking the user to double-press.
8. **State the maximum `--limit` in each list command's help** (Nice-to-have), and reject a larger value with a message that names it. *Why it matters:* `insights list --limit 200` failed with only "Bad Request", which broke our review page until we found the limit of 100.

AI tools and learning

- **Which AI tools have you leveraged while working on this project?** Claude Code
- **Level of learning derived from the project:** Significant
- **Did you gain AI value that you can use in your career?** Yes